

Java Interview Questions

Complete Answers with Definitions & Examples

This document covers **all questions** from your list, grouped by topic and de-duplicated. Each answer includes a clean definition, the key points an interviewer expects, comparison tables where useful, and short code examples for programming questions. Topics covered: Java 8/17/21 features, OOP concepts, Strings, Collections, HashMap behaviour, Streams, Functional Interfaces & Lambdas, Comparable vs Comparator, equals() & hashCode(), Exception Handling, Multithreading, Overloading vs Overriding, and core programming questions.

#	Section
1	Java Version Features (Java 8, 17, 21)
2	OOP Concepts (Polymorphism, Overloading vs Overriding, etc.)
3	Access Modifiers & static Keyword
4	Object Class & java.util Package
5	String, StringBuilder, StringBuffer & String Pool
6	Collections Framework (List, Set, Map)
7	HashMap — Internal Working & Duplicate Keys
8	Comparable vs Comparator
9	equals() and hashCode() Contract
10	Streams — Intermediate, Terminal, map vs flatMap
11	Functional Interfaces & Lambda Expressions
12	Exception Handling (try/catch/finally, throw/throws)
13	Multithreading & Synchronization
14	Data Structures
15	Programs (String reversal, Char count, Anagram, Sorts)

1. Java Version Features (Java 8, 17, 21)

Q1. Major features introduced in Java 8

Java 8 (2014) is the biggest release in modern Java because it introduced functional-style programming. Key features:

- **Lambda expressions** – concise way to represent an anonymous function: $(a,b) \rightarrow a+b$.
- **Functional interfaces** – interfaces with exactly one abstract method, annotated with `@FunctionalInterface`.
- **Stream API** – process collections in a declarative, pipeline style (filter, map, reduce, collect).
- **Method & Constructor references** – `String::length`, `ArrayList::new`.
- **Default and static methods in interfaces** – allow interfaces to evolve without breaking implementers.
- **Optional<T>** – container to avoid `NullPointerException`.
- **New Date & Time API** (*java.time*) – `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Duration`, `Period` — immutable and thread-safe.
- **CompletableFuture** – non-blocking asynchronous programming.
- **Nashorn JavaScript engine** (later removed in Java 15).
- **PermGen removed** → replaced by `Metaspace`.

Q2. Major features introduced in Java 17 (LTS)

Java 17 is a Long Term Support (LTS) release. Highlights — many of which were previewed earlier and finalised here:

- **Sealed Classes** – restrict which classes can extend/implement a class or interface using *sealed*, *permits*, *non-sealed*.
- **Records (finalised in 16)** – compact, immutable data carrier classes: `record Point(int x, int y) {}`.
- **Pattern Matching for instanceof** – `if (obj instanceof String s) { use(s); }`.
- **Switch Expressions** – switch returns a value, supports arrow syntax and yield.
- **Text Blocks** – multi-line string literals using triple quotes `"""..."""`.
- **Helpful NullPointerException messages** – JVM tells you which variable was null.
- **New macOS rendering pipeline**, strong encapsulation of JDK internals.
- **Removed**: Applet API, Security Manager (deprecated), RMI Activation.

Q3. Major features introduced in Java 21 (LTS)

Java 21 (Sept 2023) is the next LTS after 17. Important additions:

- **Virtual Threads (Project Loom)** – lightweight threads managed by the JVM; thousands or millions can be created cheaply. Use `Thread.ofVirtual().start(...)`.
- **Pattern Matching for switch** – switch can pattern-match on type and structure.
- **Record Patterns** – deconstruct records inside instanceof/switch: `if (o instanceof Point(int x, int y))`.
- **Sequenced Collections** – new interfaces `SequencedCollection`, `SequencedSet`, `SequencedMap` with first/last access methods.
- **String Templates (preview), Scoped Values (preview), Structured Concurrency (preview)**.
- **Generational ZGC** – improved garbage collection.
- **Deprecated**: 32-bit Windows port.

2. OOP Concepts (Polymorphism, Overloading vs Overriding, etc.)

Q4. What are the OOPS concepts in Java?

OOP (Object Oriented Programming) organises code around **objects** (data + behaviour). Java is built on four pillars:

- **Encapsulation** – hiding internal state by making fields private and exposing them via getters/setters. Protects invariants.
- **Inheritance** – a class acquires properties/behaviour from another class using *extends*. Promotes code reuse.
- **Polymorphism** – one name, many forms. Same method/operator behaves differently based on object type or parameters.
- **Abstraction** – exposing **what** an object does, hiding **how**. Achieved via abstract classes and interfaces.

Q5. What is Polymorphism?

Polymorphism means the ability of one entity (method, object, or reference) to take many forms. Two types in Java:

- **Compile-time polymorphism (Static)** – achieved by **method overloading**. Method to call is decided at compile time.
- **Runtime polymorphism (Dynamic)** – achieved by **method overriding**. The actual method called depends on the object's runtime type (dynamic dispatch).

```
Animal a = new Dog();    // reference: Animal, object: Dog
a.sound();               // calls Dog's overridden sound() at runtime
```

Q6. Difference between Method Overloading and Method Overriding

Aspect	Overloading	Overriding
Definition	Same method name with different parameters.	Subclass inherits a method from the parent class with the same name.
Polymorphism	Compile-time (static).	Runtime (dynamic).
Parameters	Must differ in number/type/order.	Must be exactly the same.
Return type	Can be different.	Same or covariant.
Access modifier	Can be anything.	Cannot be more restrictive than parent.
static methods	Can be overloaded.	Cannot be overridden (only hidden).
private/final	Allowed.	Cannot override.

Q7. Can we override static methods in a class or interface?

No. Static methods belong to the class, not to instances, so they are resolved at **compile time** based on the reference type. If a subclass declares a static method with the same signature, it is called **method hiding**, not overriding. Polymorphism does not apply. The same rule holds for static methods in interfaces (Java 8+): they cannot be overridden by implementing classes — you can only call them via the interface name.

3. Access Modifiers & static Keyword

Q8. Types of access modifiers in Java

Modifier	Same class	Same package	Subclass (diff pkg)	Anywhere
private	Yes	No	No	No
default (no modifier)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Q9. Use of the static keyword

static means the member belongs to the **class** itself, not to any instance. There is exactly one copy shared by all instances. It can be used with:

- **static variable** – class-level variable, shared by all objects (e.g. counter).
- **static method** – can be called without creating an object: *Math.max(...)*. It cannot access instance fields/methods directly.
- **static block** – executed once when the class is loaded; used for initialising static fields.
- **static nested class** – an inner class that does not need an enclosing instance.
- **static import** – import static members of a class so they can be used without qualification.

Q10. What are static objects / class-level members?

A "static object" usually means a reference variable declared *static* that holds a single shared object for the entire class. It is loaded into the **method area** of the JVM at class loading time and persists until the class is unloaded. Common uses: singletons, loggers, configuration holders, and shared constants. Caution in multi-threaded code — shared static state must be synchronized or made immutable to avoid race conditions.

4. Object Class & java.util Package

Q11. In which package does the Object class reside?

The **Object** class is in the **java.lang** package, which is automatically imported into every Java source file. Every class in Java implicitly extends *java.lang.Object* if no explicit superclass is specified, making it the root of the class hierarchy.

Q12. Methods available in the Object class

Method	Purpose
equals(Object o)	Logical equality check; default uses == (reference equality).
hashCode()	Returns an integer hash code; required for use in hash-based collections.
toString()	String representation of the object. Default: ClassName@hexHashCode.
getClass()	Returns the runtime Class object.
clone()	Creates and returns a copy of the object (protected; implement Cloneable).
finalize()	Called by GC before object reclamation (deprecated since Java 9).
wait() / wait(long)	Releases the monitor and waits until notified (inter-thread communication).
notify() / notifyAll()	Wake one / all threads waiting on the monitor.

Q13. What are the classes in java.util package?

java.util holds the Collections Framework and utility classes. Important members:

- **Collection interfaces:** List, Set, Queue, Deque, Map (Map is not a Collection).
- **List implementations:** ArrayList, LinkedList, Vector, Stack.
- **Set implementations:** HashSet, LinkedHashSet, TreeSet.
- **Map implementations:** HashMap, LinkedHashMap, TreeMap, Hashtable, WeakHashMap.
- **Queue/Deque:** PriorityQueue, ArrayDeque.
- **Utility classes:** Collections, Arrays, Objects, Optional.
- **Date/Time legacy:** Date, Calendar, TimeZone (largely replaced by java.time).
- **Others:** Scanner, Random, UUID, Properties, StringTokenizer, Iterator, Comparator.

Q14. Difference between Collection interface and Collections class

Collection (interface)	Collections (class)
Root interface of the framework — declares operations like add(), remove(), size(), etc.	Only class in java.util package that provides static helper methods.
Implemented by List, Set, Queue, etc.	Cannot be instantiated; methods called as Collections.sort(list), Collections.max(list), etc.
Defines behaviour.	Provides algorithms: sort, reverse, shuffle, min/max, binarySearch, synchronous transfers, etc.

5. String, StringBuilder, StringBuffer & String Pool

Q15. Why is String immutable in Java?

A String object cannot be changed after creation. Any operation that appears to modify it (e.g. *concat*, *replace*) returns a new String. Reasons:

- **String pool** – the JVM reuses identical literals; mutability would corrupt other references.
- **Security** – class loading, file paths, network URLs, DB connection strings rely on Strings; immutability prevents tampering.
- **Thread safety** – immutable objects can be safely shared between threads without synchronization.
- **HashCode caching** – String caches its hashCode; safe because contents never change. This makes HashMap lookups fast.

```
String s1 = "Hello";
String s2 = s1;
s1 = s1 + " World";    // new object created; s2 still points to "Hello"
```

Q16. String Constant Pool

The **String Constant Pool (SCP)** is a special memory region inside the heap that stores unique String literals. When you write *String s = "abc"*, the JVM first checks the pool; if the literal exists, the reference is reused, otherwise a new object is added. *new String("abc")* always creates a new object in the heap (outside the pool), but *intern()* can put it back into the pool.

```
String a = "java";
String b = "java";           // same reference as a (pool)
String c = new String("java"); // new heap object
System.out.println(a == b);   // true
System.out.println(a == c);   // false
System.out.println(a == c.intern()); // true
```

Q17. Difference between String, StringBuilder and StringBuffer

Aspect	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread safety	Thread-safe (immutable)	Not thread-safe	Thread-safe (synchronized methods)
Performance	Slow for many edits (creates new object)	Fastest (no lock)	Slower than StringBuilder due to locks
Since	JDK 1.0	JDK 5	JDK 1.0
When to use	Few changes / constants	Single string manipulation in single thread	Single string manipulation across threads

Q18. Difference between equals() and == for Strings

== compares **references** — whether two variables point to the same object in memory. **equals()** (overridden in String) compares **character content**.

```
String a = "hi";
String b = new String("hi");
a == b           // false (different objects)
a.equals(b)      // true  (same content)
```

6. Collections Framework (List, Set, Map)

Q19. What is the Collections Framework / Collection interface?

The **Collections Framework** is a unified architecture in *java.util* for storing and manipulating groups of objects. It consists of: **interfaces** (Collection, List, Set, Queue, Map), **implementations** (ArrayList, HashMap, ...) and **algorithms** (sort, search via Collections class). The **Collection interface** is the root for List/Set/Queue and defines the common contract: add(), remove(), size(), contains(), iterator(), stream()).

Q20. When to use List, Set and Map?

Use	Choose	Why
Ordered, duplicates allowed, indexed access	List (ArrayList / LinkedList)	Maintains insertion order, allows duplicates.
Unique elements, no duplicates	Set (HashSet / LinkedHashSet / TreeSet)	Enforces uniqueness via equals()/hashCode() or compareTo()
Key-value pairs	Map (HashMap / LinkedHashMap / TreeMap)	Fast retrieval by key (Map is not a Collection).

Q21. When to use ArrayList vs LinkedList?

Aspect	ArrayList	LinkedList
Internal structure	Resizable dynamic array	Doubly-linked list (also implements Deque)
Random access (get by index)	O(1) – fast	O(n) – slow
Insertion / deletion in the middle	O(n) – shifts elements	O(1) once you have the reference
Memory overhead	Lower (just an array)	Higher (each node stores prev/next pointers)
When to use	Read-heavy workloads, random access	Frequent insertions/removals at ends or middle

Q22. When to use HashSet vs TreeSet vs LinkedHashSet?

- **HashSet** – unordered, fastest (O(1) average). Backed by HashMap.
- **LinkedHashSet** – maintains insertion order; slightly slower than HashSet.
- **TreeSet** – sorted (natural order or Comparator); O(log n); backed by red-black tree.

7. HashMap — Internal Working & Duplicate Keys

Q23. How does HashMap work internally?

HashMap stores key-value pairs in an array of **buckets** (a `Node[]` table). Steps to `put(k, v)`:

- Compute `hash = hash(k.hashCode())` – a secondary hash to spread bits.
- Find the bucket index: `index = (n - 1) & hash` where `n` is array size.
- If the bucket is empty, place a new Node there.
- If the bucket has nodes, walk through the linked list comparing keys via `equals()`: if a match is found, the old value is **replaced**; otherwise the new node is appended.
- Since Java 8: when a single bucket contains more than 8 entries and the table size ≥ 64 , the linked list is converted to a **balanced (red-black) tree** to keep lookups $O(\log n)$ instead of $O(n)$.
- Default capacity 16, load factor 0.75 → resizes (doubles) when threshold is crossed.

Q24. What happens when you put a duplicate key in a Map?

If a Map already contains the key, calling `put(key, newValue)` **replaces (overrides) the existing value** and returns the old value. The number of entries does not change. Example:

```
Map<Integer,String> m = new HashMap<>();
m.put(101, "Reshmi");
m.put(101, "Nikhitha"); // returns "Reshmi", value becomes "Nikhitha"
m.put(102, "Nikhitha"); // new entry added
System.out.println(m); // {101=Nikhitha, 102=Nikhitha}
```

Note: Inserting a different key (102) simply adds a new entry. Duplicate **values** are allowed in a Map; only keys must be unique.

Q25. How to add a duplicate key WITHOUT overriding the previous value?

Since a key must be unique, you have two options:

- **Use Map<K, List<V>>** – store all values under that key. `map.computeIfAbsent(101, k -> new ArrayList<>()).add("Nikhitha");`
- **Use Multimap** from Guava or Apache Commons (e.g. `ArrayListMultimap`).
- **putIfAbsent(key, value)** – inserts only if the key is missing; does nothing if it exists.

Q26. Difference between HashMap and ConcurrentHashMap

Aspect	HashMap	ConcurrentHashMap
Thread safety	Not synchronized — unsafe in multi-threaded use	Thread-safe; designed for concurrency
null keys/values	Allows one null key, multiple null values	Does NOT allow null keys or null values
Locking	No locking	Fine-grained locking / CAS on bucket nodes (Java 8+); no global lock
Performance (single threaded)	Faster — no overhead	Slightly slower
Iterator	Fail-fast (<code>ConcurrentModificationException</code>)	Fail-safe / weakly consistent
When to use	Single-threaded code or external synchronization	Multi-threaded reads/writes with high concurrency

8. Comparable vs Comparator

Q27. Difference between Comparable and Comparator

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Where defined	Implemented inside the class being compared (internally)	Defined externally by user (any number of orderings possible)
Sort call	Collections.sort(list)	Collections.sort(list, comparator)
Modify source class?	Yes — class must implement Comparable	No — class is untouched
Functional interface?	Yes (single method)	Yes — used heavily with lambdas / Comparator.comparing(...)

```
// Comparable
class Emp implements Comparable<Emp> {
    int id; String name;
    public int compareTo(Emp o) { return this.id - o.id; }
}

// Comparator (multiple orderings)
Comparator<Emp> byName = (a, b) -> a.name.compareTo(b.name);
list.sort(Comparator.comparing(Emp::getName).thenComparingInt(Emp::getId));
```

9. equals() and hashCode() Contract

Q28. Rules / contract between equals() and hashCode()

- If *a.equals(b)* is true, then *a.hashCode() == b.hashCode()* **must** be true.
- If *a.hashCode() == b.hashCode()*, it does **NOT** guarantee that *a.equals(b)* is true (this is called a **hash collision**).
- If you override *equals()*, you **must** override *hashCode()* — otherwise hash-based collections (HashMap, HashSet) will behave incorrectly.
- *hashCode()* must consistently return the same integer for the same object during execution, provided the fields used in *equals()* do not change.

Q29. If two objects have the same hashCode, are they equal by equals()?

Not necessarily. Two unequal objects can produce the same hashCode — this is a collision. The hashCode is a hint about which bucket to look in; equality is then confirmed by *equals()*.

Q30. If two objects are equal by equals(), is their hashCode the same?

Yes — this is mandated by the contract. If your class violates this, it will break inside HashMap/HashSet.

10. Streams — Intermediate, Terminal, map vs flatMap

Q31. What is the Stream API in Java 8?

A **Stream** is a sequence of elements supporting sequential and parallel aggregate operations. It is **not a data structure** — it does not store elements; it lazily computes them on demand from a source (Collection, array, I/O channel). Streams support a **pipeline**: *source* → 0+ *intermediate ops* → 1 *terminal op*.

Q32. Intermediate vs Terminal operations

Intermediate (lazy, returns Stream)	Terminal (eager, triggers execution)
filter(Predicate)	forEach(Consumer)
map(Function)	collect(Collector)
flatMap(Function)	reduce(BinaryOperator)
sorted() / sorted(Comparator)	count()
distinct()	min() / max()
limit(n) / skip(n)	findFirst() / findAny()
peek(Consumer)	anyMatch / allMatch / noneMatch
mapToInt / mapToLong / mapToDouble	toArray() / toList()

Note: Intermediate ops are **lazy** — they only execute when a terminal op is invoked. Without a terminal operation, nothing runs.

Q33. Is forEach() intermediate or terminal?

forEach() is a **terminal** operation. It consumes the stream and produces a side effect (it returns void). Once forEach is called, the stream is closed.

Q34. Difference between map() and flatMap()

Aspect	map()	flatMap()
Purpose	Transforms each element to exactly one element.	Transforms each element into a stream, then flattens all stream elements.
Signature	Stream<R> map(Function<T, R> f)	Stream<R> flatMap(Function<T, Stream<R>> f)
Output cardinality	1 → 1	1 → N (flattens)
Typical use	Convert types (String → Integer).	Flatten nested structures (List<List<T>> → Stream<T>)

```
List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));
nested.stream().map(l -> l.size());           // [2, 2]
nested.stream().flatMap(List::stream);        // [1, 2, 3, 4]
```

Q35. Use of filter()

filter(Predicate) is an intermediate operation that keeps only elements matching the predicate (boolean condition) and discards the rest.

```
List<Integer> evens = nums.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

11. Functional Interfaces & Lambda Expressions

Q36. What is a Functional Interface?

A **functional interface** is an interface with exactly **one abstract method** (SAM – Single Abstract Method). It can have any number of default/static methods. Marked optionally with the `@FunctionalInterface` annotation, which tells the compiler to enforce the single-abstract-method rule. Lambda expressions and method references work only with functional interfaces.

```
@FunctionalInterface
interface Calculator {
    int apply(int a, int b);          // single abstract method
    default int square(int x){return x*x;} // allowed
}
Calculator add = (a, b) -> a + b;    // lambda binds here
```

Q37. Built-in Functional Interfaces (java.util.function)

Interface	Abstract method	Use
Predicate<T>	boolean test(T t)	Boolean condition (filter).
Function<T,R>	R apply(T t)	Transform $T \rightarrow R$ (map).
Consumer<T>	void accept(T t)	Perform action; returns nothing (forEach).
Supplier<T>	T get()	Produce a value with no input (lazy values).
BiFunction<T,U,R>	R apply(T,U)	Two inputs, one output.
UnaryOperator<T>	T apply(T t)	$T \rightarrow T$ (special case of Function).
BinaryOperator<T>	T apply(T,T)	Reduce two T into one T.

Q38. Lambda expression — definition and example

A **lambda expression** is a short, anonymous function that can be passed as an argument. Syntax: *(parameters) -> expression* or *(parameters) -> { statements; }*. It implements the single abstract method of a functional interface.

```
Runnable r = () -> System.out.println("hi");
Comparator<String> byLen = (a, b) -> a.length() - b.length();
list.forEach(x -> System.out.println(x));
```

Q39. What is a Record (Java 16+)?

A **record** is a special, compact form of class designed to hold immutable data. The compiler auto-generates: a canonical constructor, private final fields, accessor methods (same name as the field, not getX), and proper *equals()*, *hashCode()*, *toString()*.

```
public record Point(int x, int y) {}
Point p = new Point(3, 4);
p.x();           // accessor (no getX)
p.toString();    // Point[x=3, y=4]
```

Q40. What is a default method in an interface?

From Java 8, an interface can declare a method with a body using the *default* keyword. This allows interfaces to evolve (add new methods) without breaking existing implementing classes. Implementing classes inherit the default behaviour and may override it.

```
interface Vehicle {
    void start();
    default void honk(){ System.out.println("Beep!"); }
}
```

12. Exception Handling

Q41. What is an exception?

An **exception** is an event that disrupts the normal flow of program execution at runtime. In Java, exceptions are objects descending from *Throwable*. They can be caught and handled with try/catch/finally.

Q42. Exception hierarchy

```
Throwable
  ■■■ Error (serious JVM problems – OutOfMemoryError, StackOverflowError)
  ■■■ Exception
    ■■■ RuntimeException (UNCHECKED – NullPointerException, ArithmeticException, etc.)
    ■■■ (Other Exception subclasses – CHECKED – IOException, SQLException, ...)
```

Q43. What is Throwable?

Throwable is the **superclass of all errors and exceptions** in Java. Only objects whose class is Throwable or a subclass can be thrown or caught. It has two main subclasses: **Error** (serious problems an application should not try to handle) and **Exception** (conditions an application might want to catch).

Q44. Checked vs Unchecked Exceptions

Aspect	Checked	Unchecked (RuntimeException)
Detection	Checked by the compiler at compile time	Detected at runtime
Must declare/handle?	Yes — try/catch or throws	No
Caused by	External factors (file missing, network down)	Programming bugs (null, bad index, divide by 0)
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

Q45. Compile-time vs Runtime exceptions

Compile-time (checked) exceptions are caught by the compiler — the code will not compile unless they are handled or declared with throws. **Runtime (unchecked) exceptions** escape compile-time checks and appear during execution; they usually indicate a bug that should be fixed rather than caught.

Q46. How to handle a checked exception

- **Try/Catch**: surround the risky code with a try block and one or more catch blocks.
- **Throws**: declare the exception in the method signature so the caller handles it.
- **Wrap and rethrow**: catch the checked exception and rethrow as a custom RuntimeException after logging.

Q47. How to handle an unchecked exception

Unchecked exceptions **can** still be caught with try/catch when appropriate (e.g. parsing user input), but the better practice is to **prevent them by writing defensive code**: null checks, bounds checks, validating inputs, using *Optional*, etc.

Q48. Difference between throw and throws

throw	throws
Used to actually throw an exception object from a method block	Used in method signature to declare that the method may throw exception

throw new IllegalArgumentException("bad");	void readFile() throws IOException { ... }
Followed by an instance of Throwable.	Followed by one or more exception class names, comma-separated.
Inside a method body.	After the parameter list of a method.

Q49. Can a method declared with throws be handled somewhere else?

Yes. *throws* only declares that the method may propagate the exception; the actual handling happens in any caller higher up the stack. The exception travels up until a matching catch block is found. If none is found, the JVM terminates the thread and prints the stack trace.

```
void readData() throws IOException { /* ... */ }    // declared

void caller() {
    try {
        readData();          // handled here
    } catch (IOException e) {
        log.error("read failed", e);
    }
}
```

Q50. Why is the finally block used?

The **finally** block always executes after try/catch — whether an exception was thrown or not, and even if a *return* or another exception occurs (the only exceptions: JVM exit via *System.exit* or fatal JVM crash). It is used to **release resources** (close files, DB connections, sockets) and clean-up tasks. Modern Java prefers *try-with-resources* which closes *AutoCloseable* resources automatically.

Q51. How to create a custom exception (e.g. `StudentNotFoundException`)

Extend *Exception* for a checked custom exception, or *RuntimeException* for an unchecked one. Provide constructors that accept a message and optionally a cause.

```
public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(String message) {
        super(message);
    }
    public StudentNotFoundException(String message, Throwable cause) {
        super(message, cause);
    }
}

// Usage
if (student == null)
    throw new StudentNotFoundException("Student id " + id + " not found");
```

13. Multithreading & Synchronization

Q52. Difference between Thread (class) and Runnable (interface)

Thread	Runnable
A class in java.lang; you extend it.	A functional interface; you implement it.
Single inheritance — class can no longer extend anything else.	Allows a class to extend something else.
Tightly couples task with thread.	Separates the task (Runnable) from the thread (Thread).
Less flexible; not preferred.	Preferred — also usable with ExecutorService and lambdas.

```
Runnable r = () -> System.out.println("task");
new Thread(r).start();
```

Q53. How to make a List or Map synchronized?

Three common approaches, in increasing order of concurrency support:

- **Collections.synchronizedList(list) / synchronizedMap(map)** – wraps the collection with synchronized methods. Iteration must be manually synchronized on the wrapper.
- **CopyOnWriteArrayList / CopyOnWriteArraySet** – best when reads vastly outnumber writes; each write creates a new copy.
- **ConcurrentHashMap** – designed for high concurrent throughput; uses fine-grained locking / CAS, not a global lock.

```
List<String> sync = Collections.synchronizedList(new ArrayList<>());
Map<String,Integer> safe = new ConcurrentHashMap<>();
```

Q54. How to stabilize a Map / List in a multi-threading environment

"Stabilize" here means make access thread-safe and avoid race conditions:

- Prefer **concurrent collections** (ConcurrentHashMap, CopyOnWriteArrayList) over manual sync.
- If you must use a non-thread-safe collection, wrap with **Collections.synchronizedXxx(...)** and synchronize during iteration.
- Make shared data **immutable** when possible (List.of, Map.of) – immutability removes the need for locks.
- Use atomic types (*AtomicInteger*, *AtomicReference*) for counters.
- For complex compound actions (check-then-act), use *computeIfAbsent*, *putIfAbsent*, or external locks (*ReentrantLock*).

Q55. Difference between a synchronized List and a synchronized Map

Both are obtained from *Collections.synchronizedXxx(...)*. The key differences mirror the underlying List vs Map: a synchronized List stores ordered elements with index access; a synchronized Map stores key-value pairs. Both wrap each method call in a lock on the wrapper object, so iteration requires explicit external synchronization. (Note: the question in the source asked about an "asynchronous Map", which is not a standard Java term — the asked-about concept is **ConcurrentHashMap**, which provides better concurrency than a synchronized Map by avoiding a single global lock.)

Q56. Can we use a synchronized block for primitive types?

No. *synchronized(obj)* requires a reference to an **object's monitor**. Primitives (int, long, double, ...) do not have monitors. If you need to synchronize on a primitive, use its wrapper or a dedicated lock object:

```
private final Object lock = new Object();
synchronized (lock) {
```

```
        counter++;  
    }
```

Note: Avoid synchronizing on Integer/Boolean wrapper instances from autoboxing — cached instances can cause two unrelated threads to lock on the same monitor unintentionally.

14. Data Structures

Q57. What is a Data Structure?

A **data structure** is a way of organising and storing data in memory so that it can be accessed and modified efficiently. The appropriate structure depends on the operations you do most: search, insert, delete, traverse, sort. Choosing the right one directly affects an algorithm's time and space complexity.

Common categories:

- **Linear:** Array, Linked List, Stack, Queue, Deque.
- **Non-linear:** Tree (binary, BST, AVL, B-tree), Graph, Heap, Trie.
- **Hash-based:** Hash Table / HashMap / HashSet.
- **Specialised:** Priority Queue, LRU Cache, Bloom filter.

15. Programs

O58. Program: Count occurrences of each character in a String

```
import java.util.*;
public class CharCount {
    public static void main(String[] args) {
        String s = "programming";
        Map<Character, Integer> map = new LinkedHashMap<>();
        for (char c : s.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        System.out.println(map);    // {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
    }
}
```

Note: Java 8 stream alternative:

```
Map<Character, Long> map = s.chars()
    .mapToObj(c -> (char)c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new,
Collectors.counting()));
```

Q59. Program: Count occurrence of a SINGLE character in a String

```
public static int countChar(String s, char target) {
    int count = 0;
    for (char c : s.toCharArray()) {
        if (c == target) count++;
    }
    return count;
}
// countChar("programming", 'r') -> 2
```

Q60. Program: Check whether two strings are anagrams

Two strings are **anagrams** if they contain the same characters with the same frequencies (order may differ). Simplest approach: sort both strings and compare.

```
import java.util.Arrays;
public class Anagram {
    static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) return false;
        char[] x = a.toLowerCase().toCharArray();
        char[] y = b.toLowerCase().toCharArray();
        Arrays.sort(x);
        Arrays.sort(y);
        return Arrays.equals(x, y);
    }
    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent"));    // true
    }
}
```

O61. Program: Reverse a String without using built-in reverse methods

```
public class ReverseString {
    public static String reverse(String s) {
        char[] ch = s.toCharArray();
        int i = 0, j = ch.length - 1;
        while (i < j) {
            char t = ch[i]; ch[i] = ch[j]; ch[j] = t;
            i++; j--;
        }
        return new String(ch);
    }
    public static void main(String[] args) {
        System.out.println(reverse("hello"));    // olleh
    }
}
```

```
}  
}
```

Q62. Program: Bubble Sort

Repeatedly compares adjacent elements and swaps them if they are in the wrong order. Largest element "bubbles" to the end on each pass. Time: $O(n^2)$, Space: $O(1)$.

```
public class BubbleSort {  
    public static void sort(int[] a) {  
        int n = a.length;  
        for (int i = 0; i < n - 1; i++) {  
            boolean swapped = false;  
            for (int j = 0; j < n - 1 - i; j++) {  
                if (a[j] > a[j+1]) {  
                    int t = a[j]; a[j] = a[j+1]; a[j+1] = t;  
                    swapped = true;  
                }  
            }  
            if (!swapped) break;    // already sorted  
        }  
    }  
}
```

Q63. Program: Merge Sort

Divide-and-conquer algorithm: split the array in half recursively, sort each half, then merge them. Time: $O(n \log n)$, Space: $O(n)$.

```
public class MergeSort {  
    public static void sort(int[] a, int l, int r) {  
        if (l < r) {  
            int m = (l + r) / 2;  
            sort(a, l, m);  
            sort(a, m + 1, r);  
            merge(a, l, m, r);  
        }  
    }  
    private static void merge(int[] a, int l, int m, int r) {  
        int[] left = java.util.Arrays.copyOfRange(a, l, m + 1);  
        int[] right = java.util.Arrays.copyOfRange(a, m + 1, r + 1);  
        int i = 0, j = 0, k = l;  
        while (i < left.length && j < right.length)  
            a[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];  
        while (i < left.length) a[k++] = left[i++];  
        while (j < right.length) a[k++] = right[j++];  
    }  
}
```

O64. Demonstrate that String is immutable

```
public class ImmutableDemo {  
    public static void main(String[] args) {  
        String s = "Hello";  
        String t = s;           // both reference same object  
        s = s.concat(" World"); // a NEW object is created  
        System.out.println(s);  // Hello World  
        System.out.println(t);  // Hello (unchanged – proves immutability)  
    }  
}
```